

DATA ENGINEERING FOUNDATIONS / MODULE 03

PySpark The 30 Functions

A working handbook. One Iceberg lakehouse, thirty functions, thirty runnable examples — and every output on these pages copied out of an actual Spark run.

AUTHOR

Surender Sara

COMPANY

**NorthBay
Solutions**

ISSUED

August 2026

COMPANION

pyspark30/

Follows Module 01 (Zero to Architecture) and Module 02 (Mindset: EMR 101)

Desk reference · keep it open

How to use this handbook

Every page is one function

Signature, arguments, an example run against real data, the **actual output**, the gotcha that costs people an afternoon, and the SQL you already know.

Nothing on these pages was written from memory. Every table of output was captured from a real run of the companion code.

Every function has runnable code

The bottom of each page names the file and block. Open it, run it, change it, break it.

Eight files, not thirty — one per family. Each block is delimited so you can run one function or the whole family.

What this is not

It is not the API docs. The Apache docs list ~400 functions with one line each; this covers 30 with everything you need to use them correctly under load.

It is also not a tutorial. Module 01 and 02 taught the architecture. This assumes you know what a shuffle is.

How the 30 were chosen

PySpark exposes roughly 600–700 functions and methods. These 30 are the ones that appear in essentially every production pipeline.

Learn them properly and you can build almost anything you'll be asked for. The rest is a lookup when the moment arrives.

FAMILY	FUNCTIONS	PAGES	FILE
1 • Shape & select	select · withColumn · cast · withColumnRenamed/drop	01–04	01_shape.py
2 • Filter & conditional	filter/where · when/otherwise · coalesce · fillna/dropna	05–08	02_filter.py
3 • Strings	trim/lower/upper · split · regexp_replace · concat_ws	09–12	03_strings.py
4 • Dates & time	to_date · date_trunc · datediff/date_add	13–15	04_dates.py
5 • Aggregate	groupBy/agg · count/countDistinct · approx_count_distinct · collect_set	16–19	05_aggregate.py
6 • Join & reshape	join · broadcast · left_anti/semi · unionByName · explode	20–24	06_join_reshape.py
7 • Windows	Window/row_number · rank/dense_rank · lag/lead/rowsBetween	25–27	07_windows.py
8 • Dedup & partitions	dropDuplicates · orderBy · repartition/coalesce	28–30	08_control.py

Raw → Bronze → Silver

Three layers, two tables, one deliberately dirty dataset. Built by one script in about forty seconds.

S3 RAW

orders_20260830.json →
order_items_20260830.csv
exactly as your app
emitted it

BRONZE

bronze_db.orders_raw →
bronze_db.order_items_raw
declared schema, audit columns,
NO cleaning

SILVER

silver_db.orders
silver_db.order_items
typed, cleaned, deduplicated

RAW

Two files on S3, partitioned by
`dt=` . One is newline-delimited

JSON with an array and a nested
struct; the other is plain CSV.
Deliberately different, because
your sources will be.

Never edited. Never deleted.
Every bug downstream is fixable
by reprocessing from here.

BRONZE

A **declared** schema — not
inferred. Plus `_src_file` and

`_bronze_at` so you can
always trace a row back to the
file it came from.

Everything is still a string. The
duplicate order is still here. The
dirt is still here. **That is the
point.**

SILVER

Regions trimmed and uppercased,
emails lowercased, nulls defaulted,
strings cast to real timestamps, and
the duplicate resolved by a window
— not by `dropDuplicates()` .

Every example in this handbook
reads from here.

WHAT 00_SETUP.PY PRINTS WHEN IT FINISHES

```
PASS silver.orders has 12 rows (13 raw - 1 duplicate)
PASS silver.order_items has 18 rows
PASS ORD-1002 kept the LATER version (refunded, not completed)
PASS region cleaned to exactly 3 values: NA / EMEA / APAC
PASS the null channel became 'unknown'
```

Build the whole lakehouse: `python 00_setup.py`

Two files on S3

One master, one child. JSON on one side, CSV on the other, because that is how it arrives.

S3://NB-LAKEHOUSE/RAW/ORDERS/DT=2026-08-30/ORDERS_20260830.JSON — THE MASTER TABLE

```
{
  "order_id": "ORD-1001",
  "customer_id": "CUST-77",
  "customer_email": " Ana.Silva@Example.COM ",
  "region": "na",
  "channel": "web",
  "order_status": "COMPLETED",
  "order_ts": "2026-08-30 14:23:11",
  "currency": "USD",
  "promo_codes": ["SUMMER10", "FREESHIP"],
  "ship_address": {"city": "Austin", "state": "TX"},
  "ingested_at": "2026-08-30 23:05:00"
}
```

← padded, mixed case
 ← region spelled 7 different ways
 ← a STRING, not a timestamp
 ← an ARRAY → explode
 ← a STRUCT → dot access
 ← the dedup tie-breaker

S3://NB-LAKEHOUSE/RAW/ORDER_ITEMS/DT=2026-08-30/ORDER_ITEMS_20260830.CSV — THE CHILD TABLE

```
order_id,line_no,sku,department,qty,unit_price,discount_pct
ORD-1001,1,SKU-GR-088,Grocery,3,4.50,0.00
ORD-1001,2,SKU-EL-201,Electronics,1,249.99,0.10
ORD-1004,1,SKU-GR-088,Grocery,-3,4.50,0.00
ORD-9999,1,SKU-XX-000,Unknown,1,10.00,0.00
```

← one order, many lines
 ← NEGATIVE qty: a return
 ← ORPHAN: no such order

Why a master/child pair and not one flat file

A single table cannot teach `join`, `broadcast`, `left_anti`, `left_semi` or fan-out honestly. A parent with line items is the shape of almost every real dataset you will meet: orders and line items, patients and encounters, accounts and transactions, sessions and events.

Twelve orders and eighteen items is small enough to hold in your head and large enough that every one of the thirty functions has something real to do. Set `SCALE_ROWS = 2_000_000` in `config.py` when you want to feel a genuine shuffle.

Twelve deliberate defects

The dataset is dirty on purpose. Here is exactly what is wrong with it and why.

THE DEFECT	WHERE	WHICH FUNCTION IT EXISTS TO TEACH
Region spelled 7 ways: <code>na NA Na apac APAC emea EMEA</code>	orders	09 trim / upper — 7 raw spellings collapse to 3
Emails padded and mixed case: <code>" Ana.Silva@Example.COM "</code>	orders	09 trim / lower — and why brackets make padding visible
One order has no email at all (ORD-1009)	orders	07 coalesce · 12 concat vs concat_ws · 17 count(*) vs count(col)
One order has a null channel (ORD-1004)	orders	08 fillna — and why <code>== None</code> never matches
Some promo_codes are <code>[]</code> , one is null	orders	24 explode vs explode_outer — 8 rows vs 14
ORD-1002 arrives twice , later version corrects completed → refunded	orders	28 dropDuplicates is arbitrary; the window is deterministic
ORD-1012 is a completed order with zero line items	orders	20 join — makes inner (17) and left (18) genuinely differ
ORD-9999 is a line item with no parent order	order_items	22 left_anti — the orphan-detection QA check
ORD-1004 has qty = -3 (a return)	order_items	06 when/otherwise — a four-way classification with a real edge case
order_ts and ingested_at are strings	both	13 to_date / to_timestamp — nothing temporal works until you convert
Two customers hold multiple orders (CUST-77, CUST-42, CUST-88, CUST-19)	orders	25-27 windows — ranking, lag/lead and running totals need repeats
Departments repeat across regions with tied revenue (constructed)	example 26	26 rank vs dense_rank — ties are invisible without a tie

This is the part most tutorials skip

Clean sample data teaches you the happy path and nothing else. Every defect above was placed deliberately so that a specific function has something honest to demonstrate — and so that when you meet the same defect in production, you have already seen what it looks like on the way in.

The most valuable one is the duplicate order. It is the difference between a pipeline that silently reports the wrong status and one that does not.

silver_db.orders

SILVER_DB.ORDERS — 12 ROWS · 13 COLUMNS · THE TABLE BEHIND ALMOST EVERY EXAMPLE

order_id	customer_id	customer_email	region	channel	order_status	order_ts	currency	promo_codes
ORD-1001	CUST-77	ana.silva@example.com	NA	web	completed	2026-08-30 14:23:11	USD	[SUMMER10, FREESHIP]
ORD-1002	CUST-42	bob@shop.io	NA	app	refunded	2026-08-30 09:02:44	USD	[]
ORD-1003	CUST-19	chen.w@mail.cn	APAC	store	completed	2026-08-30 22:41:05	SGD	[WELCOME5]
ORD-1004	CUST-77	ana.silva@example.com	NA	unknown	refunded	2026-08-30 16:10:00	USD	[SUMMER10]
ORD-1005	CUST-88	dana@corp.de	EMEA	web	completed	2026-08-30 07:55:12	EUR	[]
ORD-1006	CUST-42	bob@shop.io	NA	app	pending	2026-08-30 18:30:00	USD	[NULL]
ORD-1007	CUST-51	eve@x.co.uk	EMEA	store	completed	2026-08-30 11:15:33	GBP	[VIP20, FREESHIP]
ORD-1008	CUST-19	chen.w@mail.cn	APAC	web	completed	2026-08-30 03:20:00	SGD	[]
ORD-1009	CUST-63	NULL	NA	app	completed	2026-08-30 13:47:29	USD	[SUMMER10]
ORD-1010	CUST-88	dana@corp.de	EMEA	web	cancelled	2026-08-30 20:05:00	EUR	[]
ORD-1011	CUST-05	frank@nordic.se	EMEA	store	completed	2026-08-30 08:00:00	EUR	[WELCOME5]
ORD-1012	CUST-51	eve@x.co.uk	APAC	web	completed	2026-08-30 19:12:00	SGD	[]

COLUMN	TYPE
order_id	string
customer_id	string
customer_email	string
region	string
channel	string
order_status	string
order_ts	timestamp

COLUMN	TYPE
currency	string
promo_codes	array<string>
ingested_at	timestamp
order_date	date
city	string from struct
state	string from struct

Note what survived the clean: **ORD-1006 still has a NULL promo_codes** and **ORD-1009 still has a NULL email**. Silver means typed, conformed and deduplicated — it does not mean every null was invented away. Deciding which nulls are defects and which are facts is a modelling decision, not a cleaning one.

silver_db.order_items

SILVER_DB.ORDER_ITEMS — 18 ROWS · 9 COLUMNS · THE CHILD TABLE

```

+-----+-----+-----+-----+-----+-----+-----+
|order_id|line_no|sku      |department|qty|unit_price|discount_pct|gross_amount|net_amount|
+-----+-----+-----+-----+-----+-----+-----+
|ORD-1001|1      |SKU-GR-088|Grocery   |3  |4.5       |0.0         |13.5        |13.5      |
|ORD-1001|2      |SKU-EL-201|Electronics|1  |249.99    |0.1         |249.99      |224.99    |
|ORD-1002|1      |SKU-AP-410|Apparel   |2  |39.0      |0.0         |78.0        |78.0      |
|ORD-1003|1      |SKU-EL-115|Electronics|1  |899.0     |0.05        |899.0       |854.05    |
|ORD-1003|2      |SKU-HO-007|Home      |4  |12.25     |0.0         |49.0        |49.0      |
|ORD-1004|1      |SKU-GR-088|Grocery   |-3 |4.5       |0.0         |-13.5       |-13.5     |
|ORD-1005|1      |SKU-BE-330|Beauty    |2  |18.75     |0.0         |37.5        |37.5      |
|ORD-1005|2      |SKU-AP-410|Apparel   |1  |39.0      |0.2         |39.0        |31.2      |
|ORD-1006|1      |SKU-EL-201|Electronics|1  |249.99    |0.0         |249.99      |249.99    |
|ORD-1007|1      |SKU-AP-522|Apparel   |3  |75.0      |0.15        |225.0       |191.25    |
|ORD-1007|2      |SKU-BE-330|Beauty    |1  |18.75     |0.0         |18.75       |18.75     |
|ORD-1007|3      |SKU-HO-007|Home      |2  |12.25     |0.0         |24.5        |24.5      |
|ORD-1008|1      |SKU-EL-115|Electronics|2  |899.0     |0.0         |1798.0      |1798.0    |
|ORD-1009|1      |SKU-GR-044|Grocery   |5  |3.2       |0.0         |16.0        |16.0      |
|ORD-1010|1      |SKU-BE-901|Beauty    |1  |55.0      |0.0         |55.0        |55.0      |
|ORD-1011|1      |SKU-HO-118|Home      |1  |129.0     |0.1         |129.0       |116.1     |
|ORD-1011|2      |SKU-GR-044|Grocery   |2  |3.2       |0.0         |6.4         |6.4       |
|ORD-9999|1      |SKU-XX-000|Unknown   |1  |10.0      |0.0         |10.0        |10.0      |
+-----+-----+-----+-----+-----+-----+-----+

```

COLUMN	TYPE	NOTE
order_id	string	joins to orders
line_no	int	1..n within an order
sku	string	SKU-XX-NNN → regexp_extract
department	string	5 values + 'Unknown'
qty	int	negative = return

COLUMN	TYPE	NOTE
unit_price	double	as supplied
discount_pct	double	0.00 - 0.20
gross_amount	double	qty × unit_price
net_amount	double	gross × (1 - discount)

Two rows to keep an eye on

ORD-1004 / line 1 has `qty = -3` and therefore `net_amount = -13.50`. Any example that sums revenue without thinking about returns will be quietly wrong.

ORD-9999 / line 1 has no parent order. It disappears from every inner join and is the entire subject of function 22.

Anatomy of a working example

Ten files. One builds the lakehouse, eight hold the thirty functions, one runs them all.

EVERY EXAMPLE FILE HAS THE SAME FIVE PARTS

```
"""
03_strings.py - Family 3 of 8: STRINGS (09-12)
09 trim / lower / upper / initcap
10 split (+ getItem)          ← 1. the contract
"""

from common import get_spark, read_table, block, show, ch
from pyspark.sql import functions as F

spark = get_spark("03_strings")          ← 2. one session
orders = read_table(spark, "silver", "orders")

# ===== 09 · trim / lower / upper ===== ← 3. one block
block("09", "trim / lower / upper", "...") per function
show(bronze.select(...), label="...")

# ===== verify ===== ← 4. assertions
check(bronze.select("region").distinct().count() == 7,
      "7 raw spellings collapse to 3")
spark.stop()                               ← 5. clean exit
```

Why blocks, not thirty files

Thirty files means thirty copies of session setup and thirty chances to get an import wrong. You would spend your time on ceremony.

The `# ===== NN · name =====` delimiters give you per-function granularity anyway: copy one block into a notebook, or run the whole family.

check() is what makes this a lab

Each file ends with five assertions on real numbers — 17 rows from an inner join, 8 rows from explode, 7 distinct customers.

Change something and re-run: if a check fails, you know exactly what you broke. Without them this would be a demo, not an exercise.

```
PASS select returns exactly the requested columns
PASS withColumn REPLACES, it does not duplicate
PASS try_cast produces NULL on failure
FAIL drop removed the column          <- exits 1
```

One flag moves you from laptop to EMR

config.py is the only file you edit. Nothing else contains a path, a bucket, or a catalog name.

1 · Laptop, no AWS, no extra JARs

`MODE = "local_parquet"`

The default. Tables are Parquet directories under

`./warehouse`. No catalog, no

Iceberg runtime, no network. **Start here.**

All thirty functions behave identically — only the DDL and MERGE syntax differ from Iceberg.

2 · Laptop, real Iceberg

`MODE = "local_iceberg"`

A local Hadoop catalog under

`./warehouse/iceberg`. Maven

fetches the Iceberg runtime once.

Now snapshots, time travel and MERGE all work — on your laptop.

3 · EMR / Glue

`MODE = "emr"`

Glue Data Catalog + S3. Change

`S3_BUCKET`, submit the same files

to EMR Serverless or a Glue 5.x job.

Not one line of example code changes.

GETTING STARTED

```
# 1. install (Spark 3.5+ or 4.x; Glue 5.1 ships Spark 3.5.6)
pip install pyspark

# 2. build the lakehouse - writes raw files, then bronze, then silver
python 00_setup.py

# 3. work through the families in order
python 01_shape.py           # functions 01-04
python 02_filter.py          # functions 05-08
# ...
python 08_control.py         # functions 28-30

# or all of it, stopping at the first failed check
./run_all.sh
```

Two settings already made for you

`session.timeZone = UTC` — see function 13 for the bug this prevents.

`shuffle.partitions = 8` — the default of 200

would give you 200 tasks over 18 rows and unreadable output. On a real cluster you want 2-4x your core count, not 8, and not 200.

A note on Spark versions

Everything here was executed on **Spark 4.2.0**. All thirty functions are stable across 3.5 and 4.x.

The one behavioural difference that matters is

`spark.sql.ansi.enabled` : false in Spark 3.5 (and

therefore Glue 5.x), true in Spark 4.0+. Functions **03** and **13** both show you what changes.

select

Choose columns. On Parquet and Iceberg this is not cosmetic — it changes how many bytes leave S3.

SIGNATURE

```
DataFrame.select(*cols) → DataFrame
cols: str | Column | list — names, expressions, or both
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
"name"	a plain column name
F.col("name")	the explicit Column object; needed for expressions
df["name"]	identical to F.col, but bound to one DataFrame — use it to disambiguate after a join
expr.alias("x")	any expression, named. Without alias you get a machine-generated column name

APPLIED TO SILVER_DB

```
orders.select(
    F.col("order_id"),           # explicit
    orders["region"],           # bracket form
    F.col("order_status").alias("status"), # renamed
    (F.col("order_id") == "ORD-1001").alias("is_first"), # expression
)
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+
|order_id|region|status  |is_first|
+-----+-----+-----+-----+
|ORD-1001|NA    |completed|true   |
|ORD-1002|NA    |refunded |false  |
|ORD-1003|APAC  |completed|false  |
+-----+-----+-----+-----+
```

The point people miss

Putting `select` early is a performance decision, not a tidiness one. Catalyst pushes the column list down into the Parquet reader, so unselected columns are **never decoded off S3**.

Run `.explain(mode="formatted")` and read the `ReadSchema` line: it lists exactly the columns Spark will read. Selecting 4 of 13 columns is a ~3x reduction in I/O before you have done any work.

EQUIVALENT SQL

```
SELECT order_id,
       region,
       order_status AS status
FROM silver_db.orders
```

RELATED

`drop` — the inverse

`selectExpr` — SQL strings

`colRegex` — pattern-matched columns

`withColumn` — add one, keep the rest

Run it: `01_shape.py · block 01`

withColumn

Add one column, or replace one by reusing its name. The normal way to clean a field.

SIGNATURE

```
DataFrame.withColumn(colName, col) → DataFrame
DataFrame.withColumns(dict) → DataFrame # Spark 3.3+
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
colName	the output name. If it already exists the column is replaced , not duplicated
col	a Column expression. A bare Python value must be wrapped in F.lit()
withColumns(dict)	many columns in one plan node — use this once you have more than about three

APPLIED TO SILVER_DB

```
(orders.select("order_id", "region")
  .withColumn("region_lc", F.lower("region"))) # NEW column
  .withColumn("region", F.concat(F.lit("R-"), "region"))) # REPLACED
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+
|order_id|region|region_lc|
+-----+-----+-----+
|ORD-1001|R-NA  |na       |
|ORD-1002|R-NA  |na       |
|ORD-1003|R-APAC|apac     |
|ORD-1004|R-NA  |na       |
+-----+-----+-----+
```

Forty chained withColumn calls will slow Catalyst down

Each call adds a nesting level to the logical plan. At thirty or forty, plan analysis itself becomes measurably slow — and on a large query it can dominate.

Use `withColumns({...})` (Spark 3.3+) or fold the expressions into a single `select()`. Same result, one plan node.

F.lit() is not optional. `withColumn("x", "silver")` raises: Spark expects a Column, and a bare string is not one.

EQUIVALENT SQL

```
SELECT order_id, region,
       LOWER(region) AS region_lc
FROM silver_db.orders
```

RELATED

`lit` — wrap a constant

`withColumns` — many at once

`withColumnRenamed` — rename only

`select` — when replacing most columns

Run it: [01_shape.py](#) · [block 02](#)

cast

Change a column's type. What a *bad* value does depends on ANSI mode — and that default changed between Spark 3.5 and Spark 4.

SIGNATURE

```
Column.cast(dataType)    → Column  # raises OR nulls, see below
Column.try_cast(dataType) → Column  # always NULL on failure
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
"int" / "double"	type as a string. double → int truncates , it does not round
"decimal(10,2)"	use decimal for money; double carries binary rounding error
"timestamp" / "date"	prefer to_timestamp() when you need an explicit format
try_cast	the same conversion, guaranteed to return NULL rather than raise

APPLIED TO SILVER_DB

```
bad = spark.createDataFrame([("42",), ("not_a_number",), (None,)], ["v"])

bad.withColumn("try_cast", F.col("v").try_cast("int")) # NULL in every version
bad.withColumn("cast",   F.col("v").cast("int"))       # depends on ANSI
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+
|v      |try_cast|
+-----+-----+
|42     |42      |
|not_a_number|NULL   |
|NULL   |NULL   |
+-----+-----+
```

cast with ANSI on -> NumberFormatException - CAST_INVALID_INPUT.

The version difference that will bite you on a migration

`spark.sql.ansi.enabled = false` → a malformed cast returns **NULL silently**. This is the default in Spark 3.x, and therefore in **AWS Glue 5.x and EMR 7.x**.

`spark.sql.ansi.enabled = true` → it **raises CAST_INVALID_INPUT** and the job fails. This is the default in **Spark 4.0+**.

So the same pipeline that quietly dropped bad rows for two years starts failing the day you upgrade. That is a good thing, but know it is coming. `try_cast` returns NULL in both, and says what you meant.

EQUIVALENT SQL

```
CAST(v AS INT)
TRY_CAST(v AS INT)
```

RELATED

`to_date / to_timestamp`

`F.try_to_number`

`schema on read` — declare types once, at ingest

See also function **13**

Run it: [01_shape.py](#) · [block 03](#)

withColumnRenamed / drop

Rename one column; remove one or many. Both are metadata-only — no data moves.

SIGNATURE

```
DataFrame.withColumnRenamed(existing, new) → DataFrame
DataFrame.drop(*cols) → DataFrame
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
withColumnRenamed	renames one column. If existing is not there it is a silent no-op
drop(*cols)	accepts many names. Dropping a column that does not exist is also silent
drop(df.col)	pass a Column to drop one specific side's column after a join

APPLIED TO SILVER_DB

```
tidy = (orders
        .withColumnRenamed("customer_email", "email")
        .withColumnRenamed("order_ts", "event_time")
        .drop("promo_codes", "ingested_at", "currency"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
before: 13 columns -> [<#x27;order_id<#x27;,, <#x27;customer_id<#x27;,, <#x27;customer_email<#x27;,, <#x27;region<#x27;,, <#x27;channel<#x27;,, <#x27;order_status<#x27;,, <#x27;event_time<#x27;,, <#x27;order_date<#x27;,, <#x27;city<#x27;,, <#x27;state<#x27;]
after : 10 columns -> [<#x27;order_id<#x27;,, <#x27;customer_id<#x27;,, <#x27;email<#x27;,, <#x27;region<#x27;,, <#x27;channel<#x27;,, <#x27;order_status<#x27;,, <#x27;event_time<#x27;,, <#x27;order_date<#x27;,, <#x27;city<#x27;,, <#x27;state<#x27;]
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
|order_id|customer_id|email|region|channel|order_status|event_time|order_date|city|state|
+-----+-----+-----+-----+-----+-----+-----+-----+
|ORD-1001|CUST-77|ana.silva@example.com|NA|web|completed|2026-08-30 14:23:11|2026-08-30|Austin|TX|
|ORD-1002|CUST-42|bob@shop.io|NA|app|refunded|2026-08-30 09:02:44|2026-08-30|Denver|CO|
|ORD-1003|CUST-19|chen.w@email.cn|APAC|store|completed|2026-08-30 22:41:05|2026-08-30|Singapore|SG|
+-----+-----+-----+-----+-----+-----+-----+-----+
```

The silence is the problem

Misspell the source name and nothing happens. No warning, no error — your column keeps its old name and a downstream step fails fifty lines later with a confusing message.

If a rename matters, assert it: `assert "email" in df.columns`. It costs one line and turns a mystery into a clear failure at the right place.

Renaming many columns? A single `select()` with aliases is clearer and cheaper than eight chained renames.

EQUIVALENT SQL

```
SELECT customer_email AS email,
       order_ts AS event_time,
       * EXCEPT (promo_codes, currency)
FROM silver_db.orders
```

RELATED

`toDF(*names)` — rename all at once

`select + alias` — rename many

`df.columns` — assert before you rely on it

Run it: [01_shape.py](#) • [Block 04](#)

filter / where

Keep rows. Two names for the identical method — pick one and be consistent across your codebase.

SIGNATURE

```
DataFrame.filter(condition) → DataFrame
DataFrame.where(condition) → DataFrame # literally an alias
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
Column expression	<code>F.col("region") == "NA"</code> — composable, refactorable, testable
SQL string	<code>"region = 'NA' AND qty > 0"</code> — shorter, and fine for simple predicates
<code>&</code> / <code> </code> / <code>~</code>	and / or / not. Each side must be in its own brackets — Python's operator precedence is against you here
<code>.isin()</code> <code>.between()</code> <code>.like()</code> <code>.rlike()</code> <code>.startswith()</code>	the predicate helpers you will reach for most

APPLIED TO SILVER_DB

```
orders.filter((F.col("region") == "EMEA") & (F.col("order_status") == "completed"))
orders.filter(F.col("order_status").isin("refunded", "cancelled"))
items.filter(F.col("qty").between(1, 2)) # inclusive both ends
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+
|order_id|region|order_status|
+-----+-----+-----+
|ORD-1005|EMEA  |completed   |
|ORD-1007|EMEA  |completed   |
|ORD-1011|EMEA  |completed   |
+-----+-----+-----+
```

Three traps, in the order people hit them

1. `and` / `or` / `not` raise `ValueError: Cannot convert column into bool`. Use `&` / `|` / `~`, with brackets round each side.
2. `col == None` matches **nothing, ever**. `NULL` is not a value, so no comparison to it is true. Use `isNull()` / `isNotNull()`.
3. Filter **before** joins, aggregations and windows. Catalyst usually pushes predicates down for you — but it cannot push through a Python UDF, and a filter on a partition column placed first is the only one that prevents I/O rather than discarding rows you already paid to read.

EQUIVALENT SQL

```
WHERE region = 'EMEA'
  AND order_status = 'completed'

WHERE order_status IN ('refunded','cancelled')

WHERE qty BETWEEN 1 AND 2
```

RELATED

`isNull` / `isNotNull`

`like` / `rlike` / `contains`

`left_semi` — filter by existence (fn 22)

`.explain()` — check PushedFilters

Run it: [02_filter.py](#) · block 05

when / otherwise

SQL's CASE WHEN. Chain as many branches as you need; the first true one wins.

SIGNATURE

```
F.when(condition, value) → Column
  .when(condition, value) # chain freely
  .otherwise(value)       # optional - omit and unmatched rows are NULL
```

ARGUMENTS

ARGUMENT WHAT IT DOES

condition	any boolean Column expression
value	a Column or a literal. Every branch must produce a compatible type or Spark will raise
otherwise	the fallback. Omit it and unmatched rows silently become NULL — occasionally what you want, usually not

APPLIED TO SILVER_DB

```
items.withColumn("band",
  F.when(F.col("qty") < 0, "return")
  .when(F.col("net_amount") >= 500, "high_value")
  .when(F.col("net_amount") >= 50, "standard")
  .otherwise("small"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+
|order_id|sku      |qty|net_amount|band      |
+-----+-----+-----+-----+
|ORD-1001|SKU-GR-088|3  |13.5      |small     |
|ORD-1001|SKU-EL-201|1  |224.99    |standard  |
|ORD-1002|SKU-AP-410|2  |78.0      |standard  |
|ORD-1003|SKU-EL-115|1  |854.05    |high value|
|ORD-1003|SKU-HO-007|4  |49.0      |small     |
|ORD-1004|SKU-GR-088|-3  |-13.5     |return    |
|ORD-1005|SKU-BE-330|2  |37.5      |small     |
|ORD-1005|SKU-AP-410|1  |31.2      |small     |
|ORD-1006|SKU-EL-201|1  |249.99    |standard  |
|ORD-1007|SKU-AP-522|3  |191.25    |standard  |
```

Order matters, and the negative row proves it

Branches are evaluated top to bottom and the **first** match wins. Move the `qty < 0` test to the bottom and ORD-1004's return (-13.50) is classified as `"small"` instead of `"return"` — still a valid program, quietly wrong numbers.

Put your **most specific** condition first, your catch-all last. When you find yourself writing six branches, ask whether a small lookup table plus a `broadcast` join would be clearer and easier to change.

EQUIVALENT SQL

```
CASE WHEN qty < 0 THEN 'return'
      WHEN net_amount >= 500 THEN 'high_value'
      WHEN net_amount >= 50 THEN 'standard'
      ELSE 'small' END
```

RELATED

- `coalesce` — null fallback (fn 07)
- `nullif / ifnull`
- `broadcast join` — for long lookups
- `F.expr("CASE WHEN ...")`

Run it: [02_filter.py](#) • [block 06](#)

coalesce

Return the first non-null argument. Completely unrelated to `DataFrame.coalesce()`, which merges partitions.

SIGNATURE

```
F.coalesce(*cols) → Column
    returns the first argument that is not NULL, or NULL if all are
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
*cols	two or more Columns or literals, in fallback order
F.lit(...)	wrap the final constant default, or Spark will look for a column with that name
types	all arguments must be mutually castable, or you get an <code>AnalysisException</code>

APPLIED TO SILVER_DB

```
orders.select("order_id", "customer_email",
    F.coalesce(
        F.col("customer_email"),                # 1st choice
        F.concat(F.col("customer_id"), F.lit("@no-email.local")), # 2nd
        F.lit("UNKNOWN")).alias("contact"))      # last resort
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+
|order_id|customer_email|contact|
+-----+-----+-----+
|ORD-1001|ana.silva@example.com|ana.silva@example.com|
|ORD-1002|bob@shop.io|bob@shop.io|
|ORD-1003|chen.w@mail.cn|chen.w@mail.cn|
|ORD-1004|ana.silva@example.com|ana.silva@example.com|
|ORD-1005|dana@corp.de|dana@corp.de|
|ORD-1006|bob@shop.io|bob@shop.io|
|ORD-1007|eve@x.co.uk|eve@x.co.uk|
|ORD-1008|chen.w@mail.cn|chen.w@mail.cn|
|ORD-1009|NULL|CUST-63@no-email.local|
|ORD-1010|dana@corp.de|dana@corp.de|
|ORD-1011|frank@nordic.se|frank@nordic.se|
|ORD-1012|eve@x.co.uk|eve@x.co.uk|
+-----+-----+-----+
```

Two functions, one name

`F.coalesce(a, b, c)` is the **null-fallback function** on this page.

`df.coalesce(8)` is the **partition-merge method** on page 30. They share a name and nothing else. Every PySpark engineer trips over this once; now you have.

Note ORD-1009 in the output: its email is NULL, so it falls through to the constructed `CUST-63@no-email.local`. The third argument is never reached here, but it guarantees the column is never null — which is what lets you make it NOT NULL downstream.

EQUIVALENT SQL

```
COALESCE(
    customer_email,
    customer_id || '@no-email.local',
    'UNKNOWN')
```

RELATED

`fillna` — bulk, by column (fn 08)

`nvl / ifnull` — two-argument form

`nullif` — the inverse

`df.coalesce(n)` — the other one (fn 30)

Run it: `02_filter.py · block 07`

fillna / dropna

Bulk null handling across many columns at once. Type-aware, which is the source of the surprise.

SIGNATURE

```
DataFrame.na.fill(value, subset=None) → DataFrame
DataFrame.na.drop(how="any", thresh=None, subset=None) → DataFrame
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
fill({"col": val})	the dict form. Use only this one — it is explicit about which column gets which default
fill(0)	fills every numeric column. Strings are ignored. This asymmetry catches people
how="any" / "all"	drop a row if <i>any</i> column is null, or only if <i>all</i> are
subset=[...]	restrict to these columns. Almost always pass this — unscoped how='any' is brutally aggressive

APPLIED TO SILVER_DB

```
holes = orders.select("order_id", "customer_email", "channel", "promo_codes")

holes.na.fill({"customer_email": "missing@unknown", "channel": "unknown"})
holes.na.drop(subset=["customer_email"]) # drops exactly 1 row
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+
|order_id|customer_email|channel|promo_codes|
+-----+-----+-----+-----+
|ORD-1001|ana.silva@example.com|web|[SUMMER10, FREESHIP]|
|ORD-1002|bob@shop.io|app|[ ]|
|ORD-1003|chen.w@mail.cn|store|[WELCOME5]|
|ORD-1004|ana.silva@example.com|unknown|[SUMMER10]|
|ORD-1005|dana@corp.de|web|[ ]|
|ORD-1006|bob@shop.io|app|[NULL]|
|ORD-1007|eve@x.co.uk|store|[VIP20, FREESHIP]|
|ORD-1008|chen.w@mail.cn|web|[ ]|
|ORD-1009|missing@unknown|app|[SUMMER10]|
|ORD-1010|dana@corp.de|web|[ ]|
|ORD-1011|frank@nordic.se|store|[WELCOME5]|
|ORD-1012|eve@x.co.uk|web|[ ]|
+-----+-----+-----+-----+
```

how="any" over every column will delete most of your data

`df.na.drop(how="any")` removes a row if **any** of its columns is null. On a wide table with optional fields that is nearly every row, and it happens silently — you just get a much smaller DataFrame and a confusing report. Always pass `subset=[...]` naming the columns that are genuinely mandatory. And note that `na.fill` will not touch an `array<string>` column at all — ORD-1006's NULL promo_codes survives. For arrays you need `coalesce(col, F.array())`.

EQUIVALENT SQL

```
-- fill
COALESCE(channel, 'unknown')

-- drop
WHERE customer_email IS NOT NULL
```

RELATED

- `coalesce` — per-column (fn **07**)
- `na.replace` — value substitution
- `F.array()` — empty-array default
- `thresh=n` — keep rows with n non-nulls

Run it: **02_filter.py · block 08**

trim / lower / upper / initcap

Normalisation. The single highest-value operation between bronze and silver, and the one juniors reach for a UDF to do.

SIGNATURE

```
F.trim(col) · F.ltrim(col) · F.rtrim(col) → Column
F.lower(col) · F.upper(col) · F.initcap(col) → Column
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
trim	removes leading and trailing whitespace only — not interior
lower / upper	full-string case folding
initcap	capitalises the first letter of each word, lowercases the rest
nulls	every one of these returns NULL for a NULL input . They never raise

APPLIED TO SILVER_DB

```
bronze.select("order_id",
  F.concat(F.lit("[", F.col("customer_email"), F.lit("]")).alias("raw_email"),
  F.lower(F.trim("customer_email")).alias("clean_email"),
  F.upper(F.trim("region")).alias("clean_region"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+
|order_id|raw_email|clean_email|raw_region|clean_region|
+-----+-----+-----+-----+
|ORD-1001|[ Ana.Silva@Example.COM ]|ana.silva@example.com|[na]|NA|
|ORD-1002|[bob@shop.io]|bob@shop.io|[NA]|NA|
|ORD-1003|[ Chen.W@mail.cn]|chen.w@mail.cn|[apac]|APAC|
|ORD-1004|[Ana.Silva@example.com]|ana.silva@example.com|[Na]|NA|
|ORD-1005|[dana@corp.de]|dana@corp.de|[emea]|EMEA|
|ORD-1006|[bob@shop.io ]|bob@shop.io|[NA]|NA|
+-----+-----+-----+-----+
```

Wrap in brackets so you can see the dirt

Trailing whitespace is invisible in every console, notebook and BI tool you will ever use. Concatenating `[` and `]` around a suspect column is the fastest diagnostic in data engineering, and it costs nothing.

In this dataset `trim + upper` collapses **seven distinct spellings of region into three**. Skip it and every

`groupBy("region")` downstream reports seven regions to a confused business user.

Spark ships ~90 string functions, all compiled to JVM bytecode. A Python UDF doing the same work costs 10-100x and blinds Catalyst. There is no reason to write one here.

EQUIVALENT SQL

```
UPPER(TRIM(region))
LOWER(TRIM(customer_email))
INITCAP(department)
```

RELATED

`\pad / rpad`

`translate` — character mapping

`regexp_replace` — interior whitespace (fn 11)

`length` — verify a trim worked

Run it: `03_strings.py · block 09`

split

String → array, then index into it. The standard way to pull a domain out of an email or a segment out of a structured key.

SIGNATURE

```
F.split(str, pattern, limit=-1) → Column<array<string>>
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
pattern	a regular expression , not a literal. To split on a dot you need "\\."
limit	maximum pieces. limit=2 splits on the first match only and keeps the rest intact
.getItem(n) / [n]	zero-indexed element access. Identical; pick one style
out of range	returns NULL , not an error. Guard with F.size() if it matters

APPLIED TO SILVER_DB

```
orders.select("order_id", "customer_email",
  F.split("customer_email", "@").alias("parts"),
  F.split("customer_email", "@").getItem(0).alias("local_part"),
  F.split("customer_email", "@")[1].alias("domain"),
  F.size(F.split("customer_email", "@")).alias("n_parts"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+customer_email+-----+local_part|domain+-----+
|order_id|customer_email|parts|local_part|domain|n_parts|
+-----+-----+-----+-----+-----+-----+
|ORD-1001|ana.silva@example.com|[ana.silva, example.com]|ana.silva|example.com|2|
|ORD-1002|bob@shop.io|[bob, shop.io]|bob|shop.io|2|
|ORD-1003|chen.w@mail.cn|[chen.w, mail.cn]|chen.w|mail.cn|2|
|ORD-1004|ana.silva@example.com|[ana.silva, example.com]|ana.silva|example.com|2|
|ORD-1005|dana@corp.de|[dana, corp.de]|dana|corp.de|2|
|ORD-1006|bob@shop.io|[bob, shop.io]|bob|shop.io|2|
+-----+-----+-----+-----+-----+-----+
```

The pattern is a regex, and size() on NULL is -1

`split(col, ".")` matches every character, because `.` is the regex wildcard. You want `split(col, "\\.")`.

Same for `|`, `+`, `*`, `(` and `[`.

Watch ORD-1009 in the output: its email is NULL, so `parts` is NULL and `n_parts` comes back as **-1**, not 0.

`F.size()` returns **-1** for a null array by design. A filter like `size(x) > 0` will happily let those rows through.

EQUIVALENT SQL

```
SPLIT(customer_email, '@')[1]
-- note: Spark SQL arrays are 0-indexed here,
-- unlike Hive's element_at (1-indexed)
```

RELATED

- `explode` — array → rows (fn 24)
- `element_at` — 1-indexed, negative OK
- `substring_index` — often simpler
- `regexp_extract` — when it is not a clean split

Run it: [03_strings.py](#) · [block 10](#)

regex_replace / regex_extract

Replace what matches, or capture what matches. The two halves of pattern work.

SIGNATURE

```
F.regex_replace(str, pattern, replacement) → Column
F.regex_extract(str, pattern, idx) → Column
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
pattern	Java regex syntax. Use Python raw strings (r"...") to keep backslashes sane
idx	0 = the whole match. 1 = the first capture group, 2 the second, and so on
no match	regex_extract returns an empty string , never NULL and never an error
\$1 \$2	backreferences in the replacement string of regex_replace

APPLIED TO SILVER_DB

```
items.select("sku",
    F.regex_replace("sku", "[^A-Za-z0-9]", "").alias("stripped"),
    F.regex_replace("sku", "^SKU-", "").alias("no_prefix"),
    F.regex_extract("sku", r"SKU-([A-Z]{2})-(\d{3})", 1).alias("grp1_dept"),
    F.regex_extract("sku", r"SKU-([A-Z]{2})-(\d{3})", 2).alias("grp2_num"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+
|sku      |stripped|no_prefix|grp1_dept|grp2_num|
+-----+-----+-----+-----+
|SKU-GR-088|SKUGR088|GR-088   |GR       |088     |
|SKU-EL-201|SKUEL201|EL-201   |EL       |201     |
|SKU-AP-410|SKUAP410|AP-410   |AP       |410     |
|SKU-EL-115|SKUEL115|EL-115   |EL       |115     |
+-----+-----+-----+-----+
```

An empty string is not a null, and your filter will not catch it

When `regex_extract` finds nothing you get `""`. So `WHERE domain IS NOT NULL` passes every failed extraction straight through, and your downstream counts include rows that matched nothing.

Test with `col != ""`, or wrap it: `F.nullif(F.regex_extract(...), F.lit(""))`.

Regex is expensive per row. If a fixed-position `substring()` or a `split()` would do the same job, use that instead — at a billion rows the difference is real.

EQUIVALENT SQL

```
REGEXP_REPLACE(sku, '[^A-Za-z0-9]', '')
REGEXP_EXTRACT(sku, 'SKU-([A-Z]{2})-', 1)
```

RELATED

- `rlike` — regex as a filter
- `regex_extract_all` — every match
- `substring` — cheaper, fixed position
- `nullif` — turn "" into NULL

Run it: `03_strings.py · block 11`

concat_ws / concat

Join columns into one string. The two behave completely differently around nulls, and that difference will surprise you exactly once.

SIGNATURE

```
F.concat_ws(sep, *cols) → Column    # SKIPS nulls
F.concat(*cols)         → Column    # NULL if ANY input is null
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
sep	the separator, as a plain Python string
*cols	column names or expressions. Non-strings are cast automatically
concat_ws	a null argument is skipped ; the rest still concatenate
concat	a single null makes the whole result NULL

APPLIED TO SILVER_DB

```
orders.select("order_id", "region", "channel", "customer_email",
              F.concat_ws(" | ", "region", "channel", "customer_email").alias("concat_ws"),
              F.concat(F.col("region"), F.lit(" | "), F.col("customer_email")).alias("concat"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+-----+-----+
|order_id|region|channel|customer_email|concat_ws|concat|
+-----+-----+-----+-----+-----+-----+
|ORD-1001|NA    |web    |ana.silva@example.com|NA | web | ana.silva@example.com|NA | ana.silva@example.com|
|ORD-1002|NA    |app    |bob@shop.io         |NA | app | bob@shop.io         |NA | bob@shop.io         |
|ORD-1003|APAC  |store  |chen.w@email.cn     |APAC | store | chen.w@email.cn     |APAC | chen.w@email.cn     |
|ORD-1004|NA    |unknown|ana.silva@example.com|NA | unknown | ana.silva@example.com|NA | ana.silva@example.com|
|ORD-1005|EMEA  |web    |dana@corp.de        |EMEA | web | dana@corp.de        |EMEA | dana@corp.de        |
|ORD-1006|NA    |app    |bob@shop.io         |NA | app | bob@shop.io         |NA | bob@shop.io         |
+-----+-----+-----+-----+-----+-----+
```

Look at ORD-1009

Its email is NULL. `concat_ws` produces `NA | app` — it skipped the null and joined the rest. `concat` produces **NULL for the entire row**.

This matters most when you are building a **surrogate key**. Use `concat` and one null field silently nulls the whole key; every row with a missing attribute collapses into a single NULL bucket on the next join, and the numbers are wrong in a way that is very hard to trace back.

For keys, use `concat_ws` with an explicit `coalesce` on each part, so a missing value becomes a visible sentinel rather than a disappearing row.

EQUIVALENT SQL

```
CONCAT_WS(' | ', region, channel, customer_email)
region || ' | ' || customer_email -- concat
```

RELATED

`coalesce` — defend each part (fn 07)

`format_string` — printf style

`array_join` — the array equivalent

`md5 / sha2` — hashed surrogate keys

Run it: `03_strings.py • block 12`

to_date / to_timestamp

String → a real temporal type. Nothing date-shaped works until you do this, and the timezone default will bite you.

SIGNATURE

```
F.to_date(col, format=None)      → Column<date>
F.to_timestamp(col, format=None) → Column<timestamp>
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
format	a Java datetime pattern, e.g. "dd/MM/yyyy". Omit it only for ISO-8601 input
bad input	same ANSI split as cast (fn 03): NULL when ansi is off, raises when on
to_date	drops the time component entirely
date_format(col, fmt)	the inverse — back to a formatted string

APPLIED TO SILVER_DB

```
spark.conf.set("spark.sql.session.timeZone", "UTC")  # do this in EVERY job

bronze.select("order_id", "order_ts",
  F.to_timestamp("order_ts").alias("as_timestamp"),
  F.to_date("order_ts").alias("as_date"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+
|order_id|order_ts      |as_timestamp      |as_date      |
+-----+-----+-----+-----+
|ORD-1001|2026-08-30 14:23:11|2026-08-30 14:23:11|2026-08-30|
|ORD-1002|2026-08-30 09:02:44|2026-08-30 09:02:44|2026-08-30|
|ORD-1003|2026-08-30 22:41:05|2026-08-30 22:41:05|2026-08-30|
|ORD-1004|2026-08-30 16:10:00|2026-08-30 16:10:00|2026-08-30|
+-----+-----+-----+-----+
```

The bug you will ship, and never get an alert for

`spark.sql.session.timeZone` defaults to the **JVM's** timezone. Your laptop is not UTC. The Glue container might be.

An EMR node might be something else again.

So `to_date(order_ts)` shifts by a few hours between environments, rows land in the **wrong day= partition**, and daily counts are quietly wrong at the midnight boundary. Nothing errors. Nothing alerts. Someone notices in a quarterly review, months later.

Pin it to UTC at the top of every job. Convert to local time only at the presentation layer, where a human will actually see it.

EQUIVALENT SQL

```
T0_DATE(order_ts)
T0_TIMESTAMP(order_ts, 'dd/MM/yyyy HH:mm:ss')
DATE_FORMAT(order_ts, 'yyyy-MM-dd')
```

RELATED

`cast("timestamp")` — no format arg

`from_utc_timestamp` / `to_utc_timestamp`

`unix_timestamp` — epoch seconds

`date_format` — the inverse

Run it: **04_dates.py · block 13**

date_trunc + extractors

Roll a timestamp up to a boundary, or pull one component out of it. Every "by week" or "by hour" report starts here.

SIGNATURE

```
F.date_trunc(format, timestamp) → Column<timestamp>
F.year/month/dayofmonth/hour/dayofweek(col) → Column<int>
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
format	'year' 'quarter' 'month' 'week' 'day' 'hour' 'minute' 'second'
return type	date_trunc keeps the timestamp type and zeroes everything below the unit
extractors	return INT . Good for grouping, wrong for sorting a label
dayofweek	1 = Sunday . Use date_format(col,'E') if you want a name

APPLIED TO SILVER_DB

```
orders.select("order_id",
  F.date_trunc("hour", "order_ts").alias("trunc_hour"),
  F.date_trunc("week", "order_ts").alias("trunc_week"),
  F.date_trunc("month", "order_ts").alias("trunc_month"))

orders.groupBy(F.hour("order_ts").alias("hour_of_day")).count()
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+
|hour_of_day|count|
+-----+-----+
|3          |1     |
|7          |1     |
|8          |1     |
|9          |1     |
|11         |1     |
|13         |1     |
|14         |1     |
|16         |1     |
|18         |1     |
|19         |1     |
|20         |1     |
```

date_trunc for grouping, date_format for labels

date_trunc returns a timestamp, so it **sorts correctly** and joins to a date dimension. This is what you group by.

date_format returns a string. "2026-08" sorts fine, but "August" and "Aug 2026" sort alphabetically — April before January. Format at the very end, for display, never as a grouping key.

date_trunc("week") starts weeks on **Monday**. If your business runs Sunday-Saturday you must adjust, and nobody will tell you that you didn't.

EQUIVALENT SQL

```
DATE_TRUNC('week', order_ts)
EXTRACT(HOUR FROM order_ts)
DATE_FORMAT(order_ts, 'yyyy-MM-dd HH')
```

RELATED

date_format — string output

weekofyear / quarter

last_day / next_day

trunc — the date-only version

Run it: [04_dates.py](#) · [block 14](#)

datediff / date_add

Date arithmetic. The rolling-window filter you will write a hundred times lives here.

SIGNATURE

```
F.datediff(end, start) → Column<int>    # end MINUS start, in days
F.date_add(start, days) · F.date_sub · F.add_months → Column<date>
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
datediff(end, start)	argument order is end first. Reversed gives you a negative number and a wrong report
date_add(col, n)	n may be negative; date_sub exists for readability
add_months	calendar-aware. Jan 31 + 1 month = Feb 28, not Mar 3
last_day / months_between	month-end, and fractional month differences

APPLIED TO SILVER_DB

```
ref = F.to_date(F.lit("2026-09-15"))

orders.select("order_id", "order_date",
              F.datediff(ref, F.col("order_date")).alias("days_before_ref"),
              F.date_add("order_date", 30).alias("refund_window_ends"),
              F.add_months("order_date", 3).alias("plus_3_months"),
              F.last_day("order_date").alias("month_end"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+-----+-----+
|order_id|order_date|days_before_ref|refund_window_ends|week before|plus_3_months|month_end|
+-----+-----+-----+-----+-----+-----+
|ORD-1001|2026-08-30|16           |2026-09-29       |2026-08-23|2026-11-30  |2026-08-31|
|ORD-1002|2026-08-30|16           |2026-09-29       |2026-08-23|2026-11-30  |2026-08-31|
|ORD-1003|2026-08-30|16           |2026-09-29       |2026-08-23|2026-11-30  |2026-08-31|
|ORD-1004|2026-08-30|16           |2026-09-29       |2026-08-23|2026-11-30  |2026-08-31|
|ORD-1005|2026-08-30|16           |2026-09-29       |2026-08-23|2026-11-30  |2026-08-31|
+-----+-----+-----+-----+-----+-----+
```

Filter with a date expression, not a computed column

`WHERE order_date >= add_months(current_date(), -3)` can be **pushed down to the file reader**, so entire partitions are never listed.

`WHERE datediff(current_date(), order_date) <= 90` computes a value for **every row first**, then filters. Same answer, orders of magnitude more I/O, and no partition pruning at all.

Always express the condition against the raw partition column. Check `PartitionFilters` in `.explain()` if you are not sure — if it is empty, you are scanning the whole table.

EQUIVALENT SQL

```
DATEDIFF('2026-09-15', order_date)
DATE_ADD(order_date, 30)
WHERE order_date >= ADD_MONTHS(CURRENT_DATE, -3)
```

RELATED

`months_between` — fractional

`current_date / current_timestamp`

`next_day` — next given weekday

`INTERVAL` — in Spark SQL

Run it: `04_dates.py` · block 15

groupBy / agg

The workhorse. Also the point at which your job stops being free: every groupBy is a shuffle across the network.

SIGNATURE

```
DataFrame.groupBy(*cols) → GroupedData
GroupedData.agg(*exprs) → DataFrame
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
groupBy(*cols)	the grouping keys. These become the shuffle hash key
agg(...)	one or more aggregate expressions. Always .alias() them or you get names like sum(net_amount)
agg without groupBy	aggregates the whole DataFrame to a single row
groupBy().pivot(c, vals)	long → wide. Always pass the value list — otherwise Spark runs an extra job to discover it

APPLIED TO SILVER_DB

```
sales = orders.filter("order_status = 'completed'").join(items, "order_id")

sales.groupBy("region", "department").agg(
    F.count("*").alias("lines"),
    F.sum("net_amount").alias("revenue"),
    F.round(F.avg("net_amount"), 2).alias("avg_line"),
    F.max("net_amount").alias("max_line"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+-----+-----+
|region|department|lines|revenue|avg_line|min_line|max_line|
+-----+-----+-----+-----+-----+-----+
|APAC  |Electronics|2    |2652.05|1326.03|854.05|1798.0|
|APAC  |Home       |1    |49.0   |49.0   |49.0   |49.0   |
|EMEA  |Apparel    |2    |222.45 |111.23 |31.2   |191.25 |
|EMEA  |Beauty     |2    |56.25  |28.13  |18.75  |37.5   |
|EMEA  |Grocery    |1    |6.4    |6.4    |6.4    |6.4    |
|EMEA  |Home       |2    |140.6  |70.3   |24.5   |116.1  |
|NA    |Electronics|1    |224.99 |224.99 |224.99 |224.99 |
|NA    |Grocery    |2    |29.5   |14.75  |13.5   |16.0   |
+-----+-----+-----+-----+-----+-----+
```

Filter before the groupBy. Never after.

The example filters to completed orders **before** the join and the aggregation. That shrinks the data that has to cross the network. Filtering afterwards produces the same numbers and shuffles everything anyway.

Spark performs a **map-side partial aggregation** first: each task builds a small hash map of key → running total and emits subtotals, not raw rows. A million rows with fifty departments becomes fifty rows before anything is sent. This is why **sum** and **count** are cheap at any scale — and why the aggregates on page 17 and 19 are not.

`spark.sql.shuffle.partitions` controls how many partitions come out. The default is **200**, chosen in 2015, and it is wrong for almost everyone.

EQUIVALENT SQL

```
SELECT region, department,
       COUNT(*) AS lines,
       SUM(net_amount) AS revenue
FROM ... GROUP BY region, department
```

RELATED

`pivot` — long to wide

`rollup / cube` — subtotals

`grouping_sets`

`agg over a Window` — no collapse (fn 27)

Run it: [05_aggregate.py](#) · [block 16](#)

count / countDistinct

Three different counts that people assume are one. The gap between two of them is often the data-quality metric you wanted.

SIGNATURE

```
F.count("*")           → rows, nulls included
F.count(col)          → NON-NULL values in that column
F.countDistinct(*cols) → exact distinct combinations
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
count("*")	counts rows. Always the row count, whatever is null
count(col)	skips nulls . A different number, and usually the more interesting one
countDistinct(a, b)	distinct <i>combinations</i> of the listed columns. Nulls are excluded
df.count()	the DataFrame method — an action that triggers a full pass. Not the same thing

APPLIED TO SILVER_DB

```
orders.groupBy("region").agg(
  F.count("*").alias("orders"),
  F.count("customer_email").alias("with_email"),
  (F.count("*") - F.count("customer_email")).alias("missing_email"),
  F.countDistinct("customer_id").alias("distinct_customers"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+-----+-----+
|region|orders|with_email|missing_email|distinct_customers|distinct_combos|
+-----+-----+-----+-----+-----+-----+
|APAC  |3     |3         |0           |2                 |2               |
|EMEA  |4     |4         |0           |3                 |3               |
|NA    |5     |4         |1           |3                 |5               |
+-----+-----+-----+-----+-----+-----+
```

Look at region NA: 5 orders, 4 emails

That subtraction is a **free completeness check**. Add `count(*) - count(col)` for your critical columns to any aggregation you are already running and you get data-quality monitoring at zero extra cost, because the shuffle has already happened.

countDistinct is the expensive one. Sums and counts collapse to subtotals before the shuffle; an exact distinct cannot, because it must see every value to know whether it has seen it before. So every raw value crosses the network and your 5000x shrink becomes 1x. One `countDistinct` can double a job's runtime. See page 18.

And beware `df.count()` sprinkled through a script as a debug print — each one re-executes the entire lineage above it.

EQUIVALENT SQL

```
COUNT(*), COUNT(customer_email),
COUNT(DISTINCT customer_id)
```

RELATED

`approx_count_distinct` (fn 18)

`sum(when(...).otherwise(0))` — conditional

count

`count_if` — Spark 3.5+

`df.summary()` — quick profiling

Run it: `05_aggregate.py` · block 17

approx_count_distinct

A HyperLogLog sketch instead of an exact set. Combinable before the shuffle, which is the entire reason it is cheap.

SIGNATURE

```
F.approx_count_distinct(col, rsd=0.05) → Column<bigint>
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
rsd	target relative standard deviation. Default 0.05 (5%). Lower is more accurate and uses more memory
rsd=0.01	1% error — a good default for dashboards. Below ~0.005 the cost approaches exact
memory	a fixed-size sketch per partition , regardless of cardinality
mergeable	two sketches combine into one. That is what makes map-side aggregation possible

APPLIED TO SILVER_DB

```
orders.agg(
    F.countDistinct("customer_id").alias("exact"),
    F.approx_count_distinct("customer_id").alias("approx_default_5pct"),
    F.approx_count_distinct("customer_id", 0.01).alias("approx_1pct_rsd"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+
|exact|approx_default_5pct|approx_1pct_rsd|
+-----+-----+-----+
|7    |7                  |7              |
+-----+-----+-----+
```

Identical at 12 rows. Wildly different at 900 million.

The output above shows the same number three times, because seven distinct customers fit comfortably inside any sketch. That is the honest result and it teaches nothing about cost — so here is what actually differs.

Exact: every raw `customer_id` must reach the reducer that owns its group. At 900 million rows that is 900 million values across the network.

Approximate: each partition builds a fixed-size sketch (a few KB) and only sketches are shuffled. The network cost stops depending on your row count entirely.

The real question is never "is it accurate enough" — it is **"does anyone act differently on 1,402,318 than on 1,402,000?"** For a dashboard, no. For a regulatory filing, yes. Choose accordingly.

EQUIVALENT SQL

```
APPROX_COUNT_DISTINCT(customer_id)
-- Presto/Athena: approx_distinct()
```

RELATED

- `percentile_approx` — same idea, quantiles
- `approx_percentile` in Spark SQL
- `hll_sketch_agg` — Spark 3.5+, storable sketches
- `countDistinct` (fn 17)

Run it: [05_aggregate.py](#) · [block 18](#)

collect_set / collect_list

Rows → an array, per group. Genuinely useful, and the most reliable way to run an executor out of memory.

SIGNATURE

```
F.collect_list(col) → Column<array>    # keeps duplicates and order
F.collect_set(col)  → Column<array>    # deduplicated, order NOT guaranteed
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
collect_list	every value, in partition order (not a sorted or deterministic order)
collect_set	distinct values only. Nulls are dropped by both
F.sort_array(...)	wrap the result when you need a deterministic order for comparison or hashing
F.size(...)	how many landed in the array — often what you actually wanted

APPLIED TO SILVER_DB

```
sales.groupBy("order_id").agg(
  F.collect_list("department").alias("departments_list"),
  F.collect_set("department").alias("departments_set"),
  F.sort_array(F.collect_set("sku")).alias("skus_sorted"),
  F.size(F.collect_set("department")).alias("n_distinct_depts"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+
|order_id|departments_list|departments_set|skus_sorted|n_distinct_depts|
+-----+-----+-----+-----+
|ORD-1001|[Grocery, Electronics]|[Grocery, Electronics]|[SKU-EL-201, SKU-GR-088]|2|
|ORD-1003|[Electronics, Home]|[Home, Electronics]|[SKU-EL-115, SKU-HO-007]|2|
|ORD-1005|[Beauty, Apparel]|[Apparel, Beauty]|[SKU-AP-410, SKU-BE-330]|2|
|ORD-1007|[Apparel, Beauty, Home]|[Apparel, Home, Beauty]|[SKU-AP-522, SKU-BE-330, SKU-HO-007]|3|
|ORD-1008|[Electronics]|[Electronics]|[SKU-EL-115]|1|
|ORD-1009|[Grocery]|[Grocery]|[SKU-GR-044]|1|
|ORD-1011|[Home, Grocery]|[Home, Grocery]|[SKU-GR-044, SKU-HO-118]|2|
+-----+-----+-----+-----+
```

There is no upper bound on the array, and it is built in one executor

These aggregates are **not combinable**, so every raw value crosses the network, and the resulting array is assembled in a **single executor's memory** for each group.

Group by `order_id` and each array holds a handful of items — fine. Group by `region` on a 900-million-row table where 60% of rows are `NA`, and one executor tries to build an array of 540 million elements. It dies, Spark retries the task, it dies again, and the stage fails after four attempts.

Before reaching for these, ask whether you want `size()` (a count), `max()` (one value), or a

`row_number() == 1` window (one row). Usually you do, and all three are cheap.

EQUIVALENT SQL

```
COLLECT_LIST(department)
COLLECT_SET(department)
-- Postgres: array_agg / array_agg(DISTINCT ...)
```

RELATED

`size / array_contains / sort_array`

`array_join` — array back to a string

`explode` — the inverse (fn 24)

`first / last` — when one value is enough

Run it: [05_aggregate.py](#) · block 19

join

Combine two DataFrames. In this dataset every join type gives a different row count, which is the point of the orphan and the item-less order.

SIGNATURE

```
DataFrame.join(other, on=None, how="inner") → DataFrame
how: inner | left | right | full | left_semi | left_anti | cross
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
on="col"	a shared name → one joined column in the output
on=[a.x == b.y]	an expression → both columns survive, and you must drop one
on=["a","b"]	multi-column join on shared names
how	inner drops unmatched on both sides; left keeps the left; full keeps everything

APPLIED TO SILVER_DB

```
o = orders.select("order_id", "region") # 12 rows
i = items.select("order_id", "sku", "net_amount") # 18 rows

for how in ["inner", "left", "right", "full"]:
    print(how, o.join(i, "order_id", how).count())
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
inner: 17 rows
left: 18 rows
right: 18 rows
full: 19 rows
orders=12 items=18 | ORD-9999 is an orphan item, ORD-1012 shipped nothing
```

```
+-----+-----+-----+
|order_id|region|sku      |net_amount|
+-----+-----+-----+
|ORD-1012|APAC  |NULL     |NULL      |
|ORD-9999|NULL  |SKU-XX-000|10.0     |
+-----+-----+-----+
```

Four numbers that are never the same, and why

inner 17 — ORD-9999 (orphan item) and ORD-1012 (order with no items) are both dropped.

left 18 — ORD-1012 is kept, with NULLs on the item side.

right 18 — ORD-9999 is kept, with NULLs on the order side.

full 19 — both survive.

A join is the only operation that can make your DataFrame **bigger**. If the right side has duplicate keys, every left row fans out. Row counts silently multiplying after a join is the most common silent bug in data engineering: check

`before` and `after` counts, and use `left_semi` (page 22) when you only meant to filter.

EQUIVALENT SQL

```
SELECT ... FROM orders o
LEFT JOIN order_items i USING (order_id)
```

RELATED

`broadcast` — kill the shuffle (fn 21)

`left_anti` / `left_semi` (fn 22)

`crossJoin` — explicit, and rarely correct

`.explain()` — which join algorithm won

Run it: `06_join_reshape.py` · block 20

broadcast

Ship the small side to every executor so the join needs no shuffle at all. The highest leverage single word in PySpark.

SIGNATURE

```
F.broadcast(df) → DataFrame    # a hint, not a copy
big.join(F.broadcast(small), "key")
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
what it does	sends the whole small DataFrame to every executor, turning a SortMergeJoin into a BroadcastHashJoin
autoBroadcastJoinThreshold	default 10 MB . Spark broadcasts automatically below this if it can estimate the size
when to be explicit	when Spark cannot estimate size — after a filter, after a UDF, or a source with no statistics
upper limit	must fit in each executor's memory, and in the driver first. Practically tens to a few hundred MB

APPLIED TO SILVER_DB

```
dim = orders.select("order_id", "region", "customer_id")

# our tables are tiny, so Spark auto-broadcasts BOTH. Turn that off to see
# the contrast - which is the situation you are in when Spark cannot size a side.
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")

items.join(dim, "order_id").explain(mode="simple")           # SortMergeJoin
items.join(F.broadcast(dim), "order_id").explain(mode="simple") # BroadcastHashJoin
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
== Physical Plan ==
      without broadcast (auto OFF)
+- SortMergeJoin [order_id#13], [order_id#0], Inner
   :- Sort [order_id#13 ASC NULLS FIRST], false, 0
   :- +- Exchange hashpartitioning(order_id#13, 8) <-- SHUFFLE
   +- Sort [order_id#0 ASC NULLS FIRST], false, 0
      +- Exchange hashpartitioning(order_id#0, 8) <-- SHUFFLE

== Physical Plan ==
      with an explicit broadcast() hint
+- BroadcastHashJoin [order_id#13], [order_id#0], Inner, BuildRight
   +- BroadcastExchange HashedRelationBroadcastMode(...) <-- no shuffle
```

Read the plan, not the docs

Two `Exchange hashpartitioning` lines become one `BroadcastExchange`. That is the whole optimisation, and it is routinely a 5-20x win for one word.

AQE often does this for you now — it can convert a sort-merge join to a broadcast at runtime once it sees the real size. Be explicit anyway when you *know* the side is small: AQE cannot help if the estimate is wrong before the first stage runs.

Broadcasting something that is *not* small is worse than not broadcasting: it is collected to the driver first. A 4 GB "small side" OOMs your driver, and the error will not mention broadcast.

EQUIVALENT SQL

```
SELECT /*+ BROADCAST(d) */ ...
FROM items i JOIN dim d USING (order_id)
```

RELATED

`autoBroadcastJoinThreshold`

`adaptive.enabled` — runtime conversion

`MERGE / SHUFFLE_HASH` hints

`bucketing` — when neither side is small

Run it: [06_join_reshape.py](#) · [block 21](#)

left_anti / left_semi

"Rows with no match" and "rows with a match". Neither adds columns, neither duplicates rows. The best data-quality tools you own.

SIGNATURE

```
df.join(other, on, "left_anti") → left rows with NO match
df.join(other, on, "left_semi") → left rows WITH a match
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
output columns	only the left side's . Nothing from other is added
row count	never exceeds the left side . Duplicate keys on the right cannot fan out
left_anti	orphan detection, missing dimensions, "what did we fail to load"
left_semi	filter by existence — SQL's WHERE EXISTS

APPLIED TO SILVER_DB

```
# which order_items have no parent order? -> your daily QA check
items.join(orders, "order_id", "left_anti")

# which orders shipped nothing?
orders.join(items, "order_id", "left_anti").select("order_id", "region", "order_status")

# filter by existence, WITHOUT fanning out
orders.join(items, "order_id", "left_semi")
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+-----+-----+-----+
|order_id|line_no|sku      |department|qty|unit_price|discount_pct|gross_amount|net_amount|
+-----+-----+-----+-----+-----+-----+-----+
|ORD-9999|1      |SKU-XX-000|Unknown   |1  |10.0     |0.0         |10.0        |10.0      |
+-----+-----+-----+-----+-----+-----+-----+
```

This is the check that should run every night

left_anti answers the question nobody asks until something is already wrong: *what did we receive that we cannot explain?* Orphan facts, missing dimension keys, late-arriving parents. Count it, alert on it, and you will find upstream breakage days before a business user does.

left_semi is the one people replace with an inner join and then wonder why their row count grew. An inner join against a table with duplicate keys **multiplies** your rows. **left_semi** cannot: it is a filter, not a join, and it stops at the first match.

Both are also faster than the equivalent **NOT IN** subquery, and unlike **NOT IN** they behave sanely when the right side contains NULLs.

EQUIVALENT SQL

```
-- left_anti
WHERE NOT EXISTS (SELECT 1 FROM orders o
  WHERE o.order_id = i.order_id)

-- left_semi
WHERE EXISTS (...)
```

RELATED

exceptAll / intersect

NOT IN — and its NULL trap

full outer + isNull — both sides at once

Great Expectations / Deequ for real DQ suites

Run it: **06_join_reshape.py · block 22**

unionByName

Stack DataFrames by **column name**. Its sibling `union()` matches by position and will silently scramble your data.

SIGNATURE

```
df.unionByName(other, allowMissingColumns=False) → DataFrame
df.union(other) → DataFrame # POSITIONAL. avoid.
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
<code>unionByName</code>	aligns on name. Column order may differ between the two sides
<code>allowMissingColumns=True</code>	the union of both schemas; absent columns become NULL
<code>union</code>	aligns on ordinal position . Same column count is all it checks
<code>duplicates</code>	neither deduplicates. Chain <code>.distinct()</code> if you need SQL's UNION semantics

APPLIED TO SILVER_DB

```
na = orders.filter("region = 'NA'").select("order_id", "region", "order_status")
emea = orders.filter("region = 'EMEA'").select("order_status", "order_id", "region")
# note: DIFFERENT column order

na.unionByName(emea) # correct
na.union(emea)      # same code path, scrambled data
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+
|order_id|region|order_status|
+-----+-----+-----+
|ORD-1001|NA    |completed   |
|ORD-1002|NA    |refunded    |
|ORD-1004|NA    |refunded    |
|ORD-1006|NA    |pending     |
+-----+-----+-----+
```

Look at the region column in that output

`union()` put `ORD-1005` in the `order_id` column position and the status where the region should be. No error.

No warning. Same column count, same types, completely wrong table.

This is one of the few PySpark bugs that produces **plausible-looking output**. It survives review, it survives testing on data where the column orders happen to match, and it fails in production the day someone adds a column to one source.

Use `unionByName` **unconditionally**. There is no case where positional matching is what you meant — and

`allowMissingColumns=True` handles schema drift between a new and an old source without you having to align them by hand.

EQUIVALENT SQL

```
SELECT order_id, region, order_status FROM ...
UNION ALL
SELECT order_id, region, order_status FROM ...
-- SQL is name-safe; the DataFrame API is not
```

RELATED

`allowMissingColumns=True`

`distinct()` — for UNION not UNION ALL

`exceptAll / intersectAll`

`mergeSchema` — on read

Run it: [06_join_reshape.py](#) · block 23

explode / explode_outer

Array → one row per element. One of them silently deletes rows and most people do not notice for months.

SIGNATURE

```
F.explode(col)      → Column  # DROPS rows with empty/null arrays
F.explode_outer(col) → Column  # KEEPS them, element is NULL
F.posexplode(col)   → (pos, col)
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
explode	an empty array or a NULL array produces zero output rows for that input row
explode_outer	produces exactly one output row with a NULL element instead
posexplode	also returns the zero-based index — use it when array order carries meaning
maps	both work on map columns too, yielding key and value columns

APPLIED TO SILVER_DB

```
orders.select("order_id", "promo_codes",
              F.explode("promo_codes").alias("promo"))    # 8 rows

orders.select("order_id", "promo_codes",
              F.explode_outer("promo_codes").alias("promo")) # 14 rows
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+
|order_id|promo_codes|promo|
+-----+-----+-----+
|ORD-1001|[SUMMER10, FREESHIP]|SUMMER10|
|ORD-1001|[SUMMER10, FREESHIP]|FREESHIP|
|ORD-1003|[WELCOME5]|WELCOME5|
|ORD-1004|[SUMMER10]|SUMMER10|
|ORD-1007|[VIP20, FREESHIP]|VIP20|
|ORD-1007|[VIP20, FREESHIP]|FREESHIP|
|ORD-1009|[SUMMER10]|SUMMER10|
|ORD-1011|[WELCOME5]|WELCOME5|
+-----+-----+-----+
```

12 orders in. 8 rows out. Six orders disappeared.

Six of our twelve orders have an empty or NULL `promo_codes`. `explode` drops all six. If you then join back to compute "revenue by promo", those six orders contribute **nothing** — your revenue total no longer matches the orders table, and the discrepancy is invisible in the code.

`explode_outer` returns all 12 orders (14 rows: 8 promo rows plus the 6 kept with `promo = NULL`). Totals reconcile.

The rule: use `explode` when the array *is* the fact you are counting. Use `explode_outer` whenever the parent row must survive — which, whenever you are going to sum money, is always.

EQUIVALENT SQL

```
LATERAL VIEW explode(promo_codes) t AS promo
LATERAL VIEW OUTER explode(promo_codes) t AS promo
-- or: CROSS JOIN UNNEST(promo_codes) in Athena
```

RELATED

`posexplode` / `posexplode_outer`

`inline` — array<struct> to columns

`collect_list` — the inverse (fn 19)

`size` / `array_contains`

Run it: [06_join_reshape.py](#) · block 24

Window + row_number

Rank rows inside a group without collapsing them. If you have written OVER (PARTITION BY ...) you already know this.

SIGNATURE

```
Window.partitionBy(*cols).orderBy(*cols) → WindowSpec
F.row_number().over(w) → Column<int>
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
partitionBy	the groups. Numbering restarts at 1 inside each one
orderBy	the ranking order. Required for row_number, rank and lag/lead
row_number()	1, 2, 3, 4 — strictly sequential, ties broken arbitrarily
difference from groupBy	a window keeps every row and adds a column. groupBy collapses rows

APPLIED TO SILVER_DB

```
from pyspark.sql.window import Window

w_rev = Window.partitionBy("region").orderBy(F.col("revenue").desc())

(dept.withColumn("rn", F.row_number().over(w_rev))
 .filter("rn <= 2"))      # top 2 departments per region
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+
|region|department|revenue|rn|
+-----+-----+-----+-----+
|APAC  |Electronics|2652.05|1|
|APAC  |Home       |49.0   |2|
|EMEA  |Apparel    |222.45 |1|
|EMEA  |Home       |140.6  |2|
|NA    |Electronics|224.99 |1|
|NA    |Grocery    |29.5   |2|
+-----+-----+-----+-----+
```

A window with no partitionBy sends every row to one executor

`Window.orderBy("order_ts")` with no `partitionBy` means Spark must establish a **global order**, so every row in the dataset is routed to a **single partition on a single executor**. Spark even warns you in the log, and everyone ignores it.

319 cores sit idle while one core processes 1.4 billion rows. Then it OOMs.

Always partition a window unless you genuinely need a global sequence — and if you think you do, question the requirement first. Almost every real need for a global row number is satisfied by `monotonically_increasing_id()`, which is unique and ascending but not gapless, and costs nothing.

EQUIVALENT SQL

```
ROW_NUMBER() OVER (
  PARTITION BY region
  ORDER BY revenue DESC)
```

RELATED

`rank / dense_rank` (fn 26)

`ntile / percent_rank`

`monotonically_increasing_id`

`dropDuplicates` — the naive version (fn 28)

Run it: [07_windows.py](#) · block 25

rank / dense_rank

Three ways to number a tie. They differ only when values repeat — which is exactly when a report goes wrong.

SIGNATURE

```
F.row_number().over(w) → 1, 2, 3, 4 # ties broken arbitrarily
F.rank().over(w)       → 1, 2, 2, 4 # ties share, then a GAP
F.dense_rank().over(w) → 1, 2, 2, 3 # ties share, NO gap
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
row_number	always unique. Use it to pick one row per group
rank	competition ranking. Two silver medals means no bronze
dense_rank	no gaps. Use it to answer "the top 3 <i>values</i> " rather than "the top 3 rows"
ntile(n) / percent_rank	bucketing and relative position

APPLIED TO SILVER_DB

```
# a deliberate tie: B and C both at 90
tied = spark.createDataFrame(
    [("NA", "A", 100.0), ("NA", "B", 90.0), ("NA", "C", 90.0), ("NA", "D", 80.0)],
    ["region", "dept", "revenue"])

tied.withColumn("row_number", F.row_number().over(w_t)) \
    .withColumn("rank", F.rank().over(w_t)) \
    .withColumn("dense_rank", F.dense_rank().over(w_t))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+-----+-----+-----+
|region|dept|revenue|row_number|rank|dense_rank|percent_rank|ntile_2|
+-----+-----+-----+-----+-----+-----+-----+
|NA     |A   |100.0  |1         |1   |1         |0.0         |1      |
|NA     |B   |90.0   |2         |2   |2         |0.333      |1      |
|NA     |C   |90.0   |3         |2   |2         |0.333      |2      |
|NA     |D   |80.0   |4         |3   |3         |1.0         |2      |
+-----+-----+-----+-----+-----+-----+-----+
```

"Top 3" is ambiguous, and the ambiguity is a bug

`row_number <= 3` gives you exactly 3 rows — but if there is a tie at third place, **which one you get is arbitrary and can change between runs**. Your report is not reproducible.

`rank <= 3` gives 4 rows here (A, B, C, then nothing at 3). `dense_rank <= 3` gives all four.

Decide which the business means and say so in code. If you need `row_number` for determinism, add a tie-breaker to the `orderBy` — a second sort column that is unique. `.orderBy(desc("revenue"), asc("dept"))` is reproducible; the version without it is not.

EQUIVALENT SQL

```
RANK() OVER (PARTITION BY region ORDER BY revenue DESC)
DENSE_RANK() OVER (...)
NTILE(4) OVER (...)
```

RELATED

`percent_rank / cume_dist`

`ntile` — quartiles, deciles

`first / last with ignorenulls`

`orderBy` tie-breakers

Run it: [07_windows.py · block 26](#)

lag / lead / rowsBetween

Reach into neighbouring rows, and accumulate across them. Period-over-period change and running totals, without a self-join.

SIGNATURE

```
F.lag(col, offset=1, default=None).over(w) → the PREVIOUS row
F.lead(col, offset=1).over(w)           → the NEXT row
w.rowsBetween(Window.unboundedPreceding, Window.currentRow)
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
lag / lead	return NULL at the edges of each partition. That is correct, not a bug
offset	how many rows back or forward. lag(col, 7) for week-over-week on daily data
rowsBetween	a frame counted in rows : unboundedPreceding, currentRow, unboundedFollowing, or an integer
rangeBetween	a frame counted in values of the orderBy column — use for true time windows

APPLIED TO SILVER_DB

```
w_time = Window.partitionBy("customer_id").orderBy("order_ts")
w_run  = w_time.rowsBetween(Window.unboundedPreceding, Window.currentRow)

spend.withColumn("running_total", F.round(F.sum("net_amount").over(w_run), 2)) \
      .withColumn("orders_so_far", F.count("*").over(w_run))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+-----+-----+-----+
|customer_id|order_id|order_ts|net_amount|running_total|orders_so_far|
+-----+-----+-----+-----+-----+-----+
|CUST-05|ORD-1011|2026-08-30 08:00:00|116.1|116.1|1|
|CUST-05|ORD-1011|2026-08-30 08:00:00|6.4|122.5|2|
|CUST-19|ORD-1008|2026-08-30 03:20:00|1798.0|1798.0|1|
|CUST-19|ORD-1003|2026-08-30 22:41:05|854.05|2652.05|2|
|CUST-19|ORD-1003|2026-08-30 22:41:05|49.0|2701.05|3|
|CUST-51|ORD-1007|2026-08-30 11:15:33|191.25|191.25|1|
```

The default frame is not what you think

An aggregate over a window with an `orderBy` but **no explicit frame** defaults to

`rangeBetween(unboundedPreceding, currentRow)` — a running total. Without an `orderBy` it defaults to the whole partition — a group total on every row.

So `sum(x).over(Window.partitionBy("c"))` and `sum(x).over(Window.partitionBy("c").orderBy("t"))` return **completely different numbers** and both look reasonable. Always state the frame explicitly.

Also: with tied timestamps `rangeBetween` includes *all* rows sharing the current value while `rowsBetween` counts positions. An order fanning out to several items shares one timestamp, so the two disagree here.

EQUIVALENT SQL

```
LAG(revenue) OVER (PARTITION BY c ORDER BY t)
SUM(amt) OVER (PARTITION BY c ORDER BY t
              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```

RELATED

`rangeBetween` — value-based frames

`first_value / last_value`

`nth_value`

`ignorenulls=True` on lag/lead

Run it: [07_windows.py](#) · [block 27](#)

dropDuplicates / distinct

Remove duplicate rows. Neither is enough on its own when "latest wins", which is almost always what you actually mean.

SIGNATURE

```
DataFrame.distinct()           → compares ALL columns
DataFrame.dropDuplicates([cols]) → compares the listed columns only
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
distinct()	two rows must match on every column. Any differing audit column defeats it
dropDuplicates([...])	keeps one arbitrary row per key. Which one is not defined and can change between runs
dropDuplicates()	with no arguments, identical to distinct()
both	trigger a shuffle. Neither is free

APPLIED TO SILVER_DB

```
# bronze holds ORD-1002 TWICE: same id, different ingested_at, DIFFERENT status
bronze.dropDuplicates(["order_id"]) # keeps one - but WHICH one?

# the deterministic pattern. This is what 00_setup.py uses.
w = Window.partitionBy("order_id").orderBy(F.col("ingested_at").desc())
deduped = (bronze.withColumn("_rn", F.row_number().over(w))
           .filter("_rn = 1").drop("_rn"))
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
+-----+-----+-----+
|order_id|order_status|ingested_at|
+-----+-----+-----+
|ORD-1002|completed   |2026-08-30 23:05:00|
|ORD-1002|REFUNDED    |2026-08-31 02:10:00|
+-----+-----+-----+

+-----+-----+-----+
|order_id|order_status|ingested_at|
+-----+-----+-----+
|ORD-1002|REFUNDED    |2026-08-31 02:10:00|
+-----+-----+-----+
```

"Arbitrary" means your pipeline is not reproducible

ORD-1002 arrives twice. The first says `completed` ; the corrected one, ingested three hours later, says `refunded` .

`dropDuplicates(["order_id"])` keeps one of them. Which one depends on partition ordering and can differ between runs on identical input. So your refund report is right on Monday and wrong on Tuesday, with no code change and no error.

The `Window + row_number() == 1` pattern is three lines longer and **says which row wins**. Use `dropDuplicates` only when the duplicate rows are genuinely interchangeable. Any time recency, source priority or completeness decides the winner, use the window. Every time.

EQUIVALENT SQL

```
-- the same idea in SQL
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY order_id
  ORDER BY ingested_at DESC) = 1
```

RELATED

`row_number` (fn 25)

`Iceberg MERGE INTO` — upsert at the table level

`dropDuplicatesWithinWatermark` — streaming

`exceptAll` — multiset difference

Run it: `08_control.py` · block 28

orderBy / sortWithinPartitions

A global sort costs a full range-partitioning shuffle. A local sort costs almost nothing. People reach for the wrong one constantly.

SIGNATURE

```
DataFrame.orderBy(*cols)           → GLOBAL order, full shuffle
DataFrame.sortWithinPartitions(*cols) → LOCAL order, no shuffle
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
orderBy / sort	aliases. Both produce a globally ordered DataFrame
F.col(c).desc()	direction. Also desc_nulls_last() — nulls sort first by default in Spark
sortWithinPartitions	each partition sorted independently. Great before a write for compression and file-skipping
cost	orderBy triggers a sampling job to pick range boundaries, then a range-partitioning shuffle

APPLIED TO SILVER_DB

```
items.orderBy("net_amount").explain(mode="simple")
items.sortWithinPartitions("net_amount").explain(mode="simple")

items.select("order_id", "sku", "net_amount") \
  .orderBy(F.col("net_amount").desc()).limit(5)
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
== Physical Plan ==                orderBy
+- Sort [net_amount#34 ASC NULLS FIRST], true, 0
   +- Exchange rangepartitioning(net_amount#34 ASC, 8) <-- SHUFFLE
      (+ a sampling job before it)

== Physical Plan ==                sortWithinPartitions
*(1) Sort [net_amount#34 ASC NULLS FIRST], false, 0
+- *(1) ColumnarToRow              <-- no Exchange at all

+-----+-----+-----+
|order_id|sku      |net_amount|
+-----+-----+-----+
|ORD-1008|SKU-EL-115|1798.0   |
|ORD-1003|SKU-EL-115|854.05   |
|ORD-1006|SKU-EL-201|249.99   |
|ORD-1001|SKU-EL-201|224.99   |
|ORD-1007|SKU-AP-522|191.25   |
```

How a global sort works, and why it needs a sampling job

Spark cannot sort a terabyte on one machine, so it **samples** the data to estimate the value distribution, picks range boundaries, routes each row to the reducer owning its band, sorts each band locally, and concatenates. Because band 1's minimum is above band 2's maximum, the result is globally ordered with no machine ever holding everything.

That sampling pass is the extra small job you see in the Spark UI before a big `orderBy`.

Never sort just to make output files look tidy. The consumer re-sorts anyway. If you want clustering for compression or file-skipping, `sortWithinPartitions` gets you most of the benefit for none of the cost.

EQUIVALENT SQL

```
ORDER BY net_amount DESC
-- Spark-only, no SQL equivalent:
SORT BY net_amount
```

RELATED

`limit` — often what you meant

`desc_nulls_last / asc_nulls_first`

`repartitionByRange`

`Iceberg sort order` — declared on the table

Run it: `08_control.py` • block 29

repartition / coalesce

The two ways to change partition count. One shuffles and can do anything; one only merges and is nearly free.

SIGNATURE

```
DataFrame.repartition(n)           → full shuffle, EXACT n, can grow
DataFrame.repartition(n, *cols)    → hash-partition by key
DataFrame.coalesce(n)              → merge only, no shuffle, can only shrink
```

ARGUMENTS

ARGUMENT	WHAT IT DOES
repartition(n)	round-robin. Even partitions, at the cost of a full shuffle
repartition(n, "col")	hash by key — co-locates all rows for a key on one partition . Use before repeated joins on that key
coalesce(n)	merges adjacent partitions in place. Cheap, but can leave them uneven
gotcha	coalesce(1) before a wide transform can collapse the whole upstream stage to one task

APPLIED TO SILVER_DB

```
items.rdd.getNumPartitions()      # as read
items.repartition(6)              # exactly 6, shuffled
items.repartition(4, "department") # each department on ONE partition
items.coalesce(1)                 # merged, no shuffle
```

OUTPUT — CAPTURED FROM THE ACTUAL RUN

```
partitions as read      : 1
after repartition(6)    : 6
after repartition by key: 4
after coalesce(1)       : 1

+---+-----+-----+
|pid|departments|rows|
+---+-----+-----+
|0  |[Home, Beauty]|6  |
|1  |[Electronics, Unknown]|5  |
|2  |[Grocery]|4  |
|3  |[Apparel]|3  |
```

coalesce(1) before a wide transformation will strangle the whole stage

`coalesce` does not add a shuffle boundary, so the reduced parallelism **propagates backwards** through the entire stage. Write `df.coalesce(1).groupBy(...)` and your aggregation runs on **one core**, not because the aggregation is small but because you told the stage it had one partition.

If you need one output file *and* parallel computation, do the work first and `coalesce` immediately before `.write`

— or use `repartition(1)`, which inserts a shuffle boundary and keeps the upstream stage parallel.

The everyday use: `coalesce()` before a write, so you emit a few 128 MB files instead of 200 tiny ones.

EQUIVALENT SQL

```
-- no direct SQL equivalent
/*+ REPARTITION(6) */
/*+ COALESCE(1) */
/*+ REPARTITION(4, department) */
```

RELATED

`spark.sql.shuffle.partitions`

`repartitionByRange`

`spark_partition_id` — inspect the split

`maxRecordsPerFile` — cap output size

Run it: `08_control.py` · block 30

Ten more worth knowing

Not in the 30, but the next ten you will reach for. Two of them are diagnostics, not transformations.

FUNCTION	WHAT IT DOES	WHEN YOU WILL WANT IT
<code>pivot(col, values)</code>	Long → wide.	Building a matrix for a report. Always pass the value list — without it Spark runs an extra job just to discover the distinct values.
<code>cache()</code> / <code>persist()</code>	Keep a materialised DataFrame in executor memory across actions.	Only when you reuse the same DataFrame 3+ times . Cached and used once means you made the job slower. Always <code>unpersist()</code> after.
<code>regexp_extract_all</code>	Every match as an array, rather than one.	Pulling all hashtags, codes or IDs out of a free-text field. Pair it with <code>explode</code> .
<code>element_at(arr, n)</code>	1-indexed array access; negative n counts from the end.	When you want the last element. <code>arr[-1]</code> does not work; <code>element_at(arr, -1)</code> does.
<code>struct()</code> / <code>col.field</code>	Build a nested struct, or reach into one with dot notation.	Our <code>ship_address.city</code> in <code>00_setup</code> . Grouping related columns before writing nested Parquet.
<code>percentile_approx</code>	Approximate quantiles, using the same sketch idea as function 18.	Medians and p95s at scale. An exact percentile is not combinable and costs you the same as an exact distinct.
<code>monotonically_increasing_id</code>	A unique, ascending 64-bit id. Costs nothing.	A surrogate key when you do not need a sequence. Do not expect 1,2,3 — values jump between partitions.
<code>input_file_name</code>	The source file each row came from.	Bronze audit columns. When a row is wrong, this is how you find the file that produced it.
<code>spark_partition_id</code>	Which partition a row currently sits in.	Diagnosing skew. Group by it and count — if one partition holds 60% of rows, you have found your slow task.
<code>count_if(cond)</code>	Count rows matching a predicate inside <code>agg()</code> . Spark 3.5+.	Replaces <code>sum(when(cond,1).otherwise(0))</code> . Same result, clearer intent.

Why these are not in the 30

Each is genuinely useful and each shows up in real pipelines — but none is something you will type every week.

`pivot` and `cache` come closest, and both are demonstrated in the companion code as bonus blocks (`05_aggregate.py` · B1 and `08_control.py` · B2).

The two diagnostics worth knowing cold

`spark_partition_id()` and `input_file_name()` are not transformations you ship — they are **instruments**. When a stage is slow, group by partition id and count. When a value is wrong, add the source file and find it. Between them they answer most "why is this happening" questions in about two minutes.

SQL → PySpark, one page

If you already write warehouse SQL, this is the whole translation. Pin it next to the cheat sheet.

SQL / REDSHIFT / SNOWFLAKE	PYSPARK	FN
SELECT a, b, c	df.select("a","b","c")	01
SELECT expr AS x	df.select(expr.alias("x"))	01
SELECT *, expr AS x	df.withColumn("x", expr)	02
CAST(a AS INT)	F.col("a").cast("int")	03
WHERE a > 10 AND b = 'x'	df.filter((F.col("a")>10) & (F.col("b")== "x"))	05
WHERE a IN (...)	df.filter(F.col("a").isin(...))	05
WHERE a IS NULL	df.filter(F.col("a").isNull())	05
CASE WHEN ... ELSE ... END	F.when(...).otherwise(...)	06
COALESCE(a, b, 'c')	F.coalesce("a","b",F.lit("c"))	07
UPPER(TRIM(a))	F.upper(F.trim("a"))	09
SPLIT_PART(a,'@',2)	F.split("a","@").getItem(1)	10
REGEXP_REPLACE / REGEXP_SUBSTR	F.regexp_replace / F.regexp_extract	11
a ' ' b	F.concat_ws(" ", "a", "b")	12
TO_DATE(a, fmt)	F.to_date("a", fmt)	13
DATE_TRUNC('week', ts)	F.date_trunc("week", "ts")	14
DATEDIFF(day, a, b)	F.datediff("b", "a") end first	15
GROUP BY a HAVING SUM(x)>0	df.groupBy("a").agg(...).filter(...)	16
COUNT(DISTINCT a)	F.countDistinct("a")	17
APPROX_COUNT_DISTINCT(a)	F.approx_count_distinct("a")	18
ARRAY_AGG(a) / LISTAGG	F.collect_list("a") / collect_set	19
LEFT JOIN b USING (k)	df.join(b, "k", "left")	20
/*+ BROADCAST(b) */	df.join(F.broadcast(b), "k")	21
WHERE NOT EXISTS (...)	df.join(b, "k", "left_anti")	22
UNION ALL	df.unionByName(other) not union()	23
LATERAL VIEW explode / UNNEST	F.explode / F.explode_outer	24
ROW_NUMBER() OVER (PARTITION BY...)	F.row_number().over(Window.partitionBy(...))	25
RANK() / DENSE_RANK()	F.rank() / F.dense_rank()	26
LAG(a) OVER (...)	F.lag("a").over(w)	27
QUALIFY ROW_NUMBER() ... = 1	.withColumn("rn",...).filter("rn=1")	28
ORDER BY a DESC	df.orderBy(F.col("a").desc())	29
DISTKEY / CLUSTER BY	df.repartition(n, "a")	30

Where these outputs came from

And the two places where running the code proved an early draft wrong.

Nothing here was written from memory

Every output table in this handbook was captured from a real Spark run of the companion code. The command that produced them is `./run_all.sh`.

Two claims in an early draft were **wrong**, and running the code is what caught them. Both corrections are now the most useful part of their pages.

Environment

Spark 4.2.0
Java OpenJDK 21
MODE local_parquet
Rows 12 orders, 18 items
Assertions 44, all passing

All 30 functions behave identically on Spark 3.5 (Glue 5.x, EMR 7.x) apart from the two ANSI differences below.

THE CLAIM AN EARLY DRAFT MADE	WHAT SPARK ACTUALLY DOES	NOW ON
"A failed cast() yields NULL, it does not raise."	True with ansi.enabled=false (Spark 3.5, Glue 5.x). False in Spark 4.0+ , where it raises CAST_INVALID_INPUT. try_cast is the version-independent answer.	Fn 03
"A non-ISO date string parses to NULL."	The same ANSI split. Under Spark 4 defaults, to_date('30/08/2026') raises rather than returning NULL.	Fn 13

Why that matters more than the handbook being tidy

A pipeline written on Glue 5.x has been silently turning malformed values into NULLs for as long as it has existed. The day it moves to a Spark 4 runtime it starts failing instead — on data it was quietly discarding all along.

That is a **good** failure. But it arrives as an unexplained outage during an upgrade, and someone spends a day on it. Now you will spend five minutes.

FILE	FUNCTIONS	CHECKS
00_setup.py	raw → bronze → silver	5
01_shape.py	01-04	5
02_filter.py	05-08	5
03_strings.py	09-12	4
04_dates.py	13-15	5
05_aggregate.py	16-19 + pivot	5
06_join_reshape.py	20-24	5
07_windows.py	25-27	5
08_control.py	28-30 + cache	5

THE WHOLE HANDBOOK, IN FOUR LINES

**Filter first.
Shuffle once.
Never a UDF.
Read the plan.**

Thirty functions will build almost any pipeline you are asked for. The other 570 are a lookup — and by the time you need one, you will know what to search for. That is the actual skill.